

1 Probabilistic Forecasting

Probabilistic forecasting moves beyond point forecasts (averages) to estimate uncertainty, predicting extremes like the 90th quantile or future volatility. Main sources of uncertainty include random error, model parameter estimates, model choice, and the consistency of the data-generating process.

Analytical Methods Assumes a normal distribution of forecasts; the prediction interval for an h-step forecast is $\hat{y} \pm c \times \sigma_h$. For a 95% interval, c is typically 1.96. Standard deviations for multi-step forecasts usually increase with the horizon.

Simulation & Bootstrapping residuals Used when normal assumptions are unreasonable. It randomly draws from past residuals to simulate future scenarios repeatedly, calculating intervals via percentiles of the realizations.

Quantile Regression Predicts specific conditional quantiles of the response rather than the mean. It utilizes "pinball loss," which penalizes under-forecasting and over-forecasting asymmetrically. Higher Quantiles place a higher penalty on under-forecasting. Lower Quantiles described as the "mirror" of the 0.95 quantile. **dif between Quantile and Bootstrapping residuals** Quantile regression directly estimates specific conditional quantiles (like the 5th and 95th) by minimizing a tilted loss function, allowing the prediction interval shape to adapt to the data's distribution without assuming normality. In contrast, bootstrapping residuals involves resampling historical forecast errors to simulate many future paths, using the empirical distribution of those paths to calculate intervals.

2 Anomaly Detection

Outliers are observations significantly different from the majority, often caused by errors or unique events. Global outliers are outside the entire dataset, while local/contextual outliers deviate from their specific temporal context.

- **Rolling Median:** Subtracts rolling median and flags values outside $\pm 1.96\sigma$; use this as a simple baseline to identify single anomalous points relative to their local temporal context.
- **STL Decomposition:** Decomposes data and uses residual quantiles ($q_{0.9} - q_{0.1}$) to flag anomalies; use this for seasonal or non-seasonal data to isolate outliers after removing trend and seasonal signals.
- **Isolation Forest:** A tree ensemble where outliers are identified closer to the root (shorter path lengths); use this for an unsupervised machine learning approach that is robust to complex data and does not assume a specific distribution.
- **K-Nearest Neighbor (KNN):** Flags points with the largest average distance to their k-neighbors; use this when you want a distance-based machine learning approach to identify observations that are contextually or spatially isolated from clusters.

3 Imputation Techniques

Imputation is the filling of missing values or outliers. Techniques include removing rows (`.dropna()`), naive fills (mean/median), rolling statistics, and polynomial interpolation. Temporal imputation uses values from the same period in past seasons.

4 Deep Learning Foundations

Classical methods like ARIMA focus on linear relationships and assume complete data. Neural networks are flexible architectures robust to noise/missing values that support non-linear mappings. Conventional feedforward networks are limited because they require "pre-assigned" lag counts and cannot model long-term dependencies.

4.1 Recurrent Neural Networks (RNN)

RNNs maintain a "hidden state" that serves as memory for past behavior. Data is passed sequentially; at each step, the model updates the hidden state using the past state and new input with a non-linearity applied. Early values influence later outputs via the recurrent structure. The same weights and biases are applied at each time step!

Backpropagation Through Time (BPTT) sums gradient contributions across steps. It is infeasible to backpropagate loss through the entire sequence. Instead: Data is typically split into chunks, backpropagated through the timesteps within a chunk; The chunks are non-overlapping. We could also decide to use overlapping chunks. We can compute the loss using only the final prediction from a chunk, or all predictions.

Stateless training resets the hidden state back to the default (e.g. all zeroes) between chunks.

Stateful training will pass the hidden state on between chunks (but

still only backpropagate the loss within the chunk). May help model to remember longer term dependencies. It uses non-overlapping chunks.

4.2 LSTMs and Gradients

RNNs suffer from vanishing/exploding gradients when learning from the distant past. Long Short-Term Memory (LSTM) models use a "cell state" to model long-term memory and allow easier gradient flow.

Implementation Details - PyTorch Setup Scaling data is a critical preprocessing step for RNNs. Inputs for RNN/LSTM cells are typically shaped as (`batch_size`, `sequence_length`, `input_size`). Models can be "stacked" to create multi-layer Deep RNNs using `num_layers` in PyTorch.

Multivariate and Attention Adapting RNNs for multivariate series only requires changing the input/output shape specifications. Attention mechanisms help circumvent the issue of encoding entire sequences into fixed-length vectors, leading to Transformers.

5 Time Series Foundational Models

Pre-trained models like Chronos or TimesFM offer "zero-shot" forecasting, making predictions without necessarily training on the specific user dataset. Hierarchical series have multiple levels (e.g., state vs. country). Bottom-up approaches model the lowest level first and sum upward, while top-down approaches model the highest level and proportion downward.

Forecasting Packages Prophet is useful for complex seasonality and holiday effects. Gluon-TS and PyTorch Forecasting provide high-level APIs for deep learning. In R, the `tidyverts` ecosystem provides similar functionality to statsmodels.

The Gold Standard Workflow 1. Plot and decompose to identify trend/seasonality. 2. Set baselines with ETS or ARIMA. 3. Move to ML/DL if more performance is needed for complex mappings. Always prefer parsimony (simplicity) when forecasting the unknown future.

6 Spatial Data Foundations

Spatial data science focuses on **wrangling, visualizing, and modeling spatial dependence**. There are two main ways to represent spatial data: **Vector format** (discrete shapes) and **Raster format** (continuous grids).

7 Vector Data & Storage

Vector data uses vertices (x,y) to define **Points** (house locations), **Lines**(roads), or **Polygons**(closed borders like lakes).It is most commonly stored in **shapefiles** which require three primary files:`.shp`(geometries), `.shx`(index), `.dbf`(attributes), `.prj`(projection information). Each shapefile can only contain one geometry type.

Geopandas— It is the primary library for vector data, built on **shapely** and **pandas**. It introduces the **GeoSeries**(storing geometries) and **GeoDataFrame**(a pandas dataframe with one or more geometry cols). 5 columns of dtype "object" which typically means "strings" in pandas, and we have our one "geometry" column which contains vector data - polygons in this case. MULTIPOLYGONS which just means multiple polygons together - provinces]"geometry"])[0]

Data Acquisition— The `osmnx` library provides an API to query **OpenStreetMap (OSM)** for real-world data like road/bike networks, building heights, and city polygons. It can return a networkx graph that can be converted to a GeoDataFrame.

7.1 Geometric Wrangling

Spatial wrangling utilizes methods like `.clip()`(cookie-cutter extraction), `.buffer()`(adding area around a geometry), and `.sjoin()`(joining datasets based on spatial location/containment).

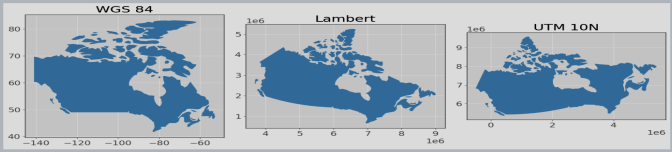
```
# Buffer to create 3m wide paths
joi=gpd.sjoin(polygons, points, how="inner", predicate="contains")
paths["geometry"] = paths.buffer(distance=1.5)
```

7.2 Coordinate Reference Systems

A CRS projects the 3D Earth onto a 2D plane. Projections are identified by **EPSG codes** **Geographic CRS** e.g.**WGS 84** / **EPSG:4326** use angular units (degrees) and are best for global positioning. **Projected CRS** e.g.,**UTM** or **Lambert** / **EPSG:3347** use linear units (meters) and are **required for calculating lengths and areas** accurately.

No projection is perfect (it's impossible to perfectly flatten a 3d sphere) and each comprises on minimizing the distortion of shapes, distances, and areas of the Earth. Different CRSs distort spatial data in different ways. So when doing spatial analysis like joining, clip-

ping, mapping, etc. It's important to have all geometries using the same CRS.



8 Raster Data

Raster data is a **matrix of pixels (cells)** where each cell represents a spatial region containing a specific value. **Resolution** indicates the ground area per pixel (e.g., 1m vs 8m); "high resolution" implies a smaller pixel size. The core Python package for raster manipulation is **rasterio**, which handles **GeoTIFF(.tif)** files containing georeferenced image data. digital image data.

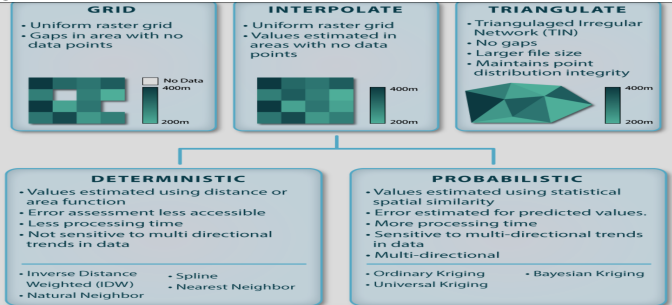
9 Spatial Visualization

Geospatial data can be visualized using `.plot()` for quick explorations and layer stacking. For interactive zoom and pan functionality, `plotly.express` functions like `choropleth_mapbox` are highly effective. **Geopandas** also offers `.explore()` to create interactive maps. Adding background tiles typically requires converting data to Web Mercator (EPSG:3857) using `contextily`.

Kepler.gl is a powerful web-based tool for spatial visualization with a Python API that supports complex configurations and 3D elements,. It uses a workflow of creating a map instance, adding data, and customizing via a graphical user interface.

10 Spatial Modeling Foundations

Spatial modeling is based on Tobler's first fundamental law of geography: "everything is related to everything else, but near things are more related than distant things",. Modeling typically involves **1.spatial interpolation** to estimate values in unobserved locations **2.areal interpolation** to project data between different sets of polygons.



10.1 The Variogram

A variogram describes how variance changes with the distance (lag) between locations, serving as the spatial analogue to the time-series ACF,. Key components include the **range** (the distance where spatial correlation stops), the **sill** (maximum variance at the range), and the **nugget** (random or measurement error at zero distance) [20].

10.2 Areal Interpolation

Areal interpolation projects values from one set of polygons (source) to another (target), such as mapping pollution data to postal codes. The most common method uses the **tobler** library to distribute values based on area proportions,.

10.3 Shortest Path Analysis

Network analysis involves finding the optimal path through a network, such as a walking or driving graph. `osmnx` is used to load these networks from OpenStreetMap, and Dijkstra's algorithm is the default method for computing the shortest path by length between an origin and destination node,.

```
route = nx.shortest_path(G, orig_id, dest_id, weight="length")
```

11 real-life applications of spatial interpolation

Environmental agencies apply spatial interpolation to estimate pollutant concentrations at unmonitored locations using data from sensor networks, helping track air quality trends over time. City planners use spatial interpolation to assess land use patterns and predict traffic flow in areas where direct data collection is sparse, aiding in infrastructure development decisions.